

Redpwn 2020 Crypto - worst pw manager

deuterium — 2020-06-27

Canonical HTML: <https://deut-erium.github.io/WriteUps/2020/redpwn/crypto/worst-pw-manager/2020-06-27-redpwn-2020-worst-pw-manager.html>

Abstract

CTF writeup: Eight Python references in a trench coat pose as an RC4 key in redpwnCTF 2020's worst-pw-manager. Decode filenames, then brute-force the repeated key byte.

worst-pw-manager

Description

I found this in-progress password manager on a dead company's website. Seems neat.

Files

- [worst-pw-manager.zip](#)
 - [worst-pw-manager/worst-pw-manager.py](#)
 - [worst-pw-manager/passwords/](#)
 - [worst-pw-manager/passwords/0'135791.enc](#)
 - ...

worst-pw-manager.py workings

```
def main(args):
    if len(args) != 2:
        print("usage: python {} [import|export|microwave_hdd]".format(args[0]))
        return

    if args[1] == "import":
        pathlib.Path("./passwords").mkdir(exist_ok=True)
        print("Importing from passwords.txt. Please wait...")
        passwords = open("passwords.txt").read()
        for pw_idx, password in enumerate(passwords.splitlines()):
            # 100% completely secure file name generation method
            masked_file_name = "".join([chr((((c - ord("0") + i) % 10) + ord("0")) *
int(chr(c) not in string.ascii_lowercase) + (((c - ord("a") + i) % 26) + ord("a")) *
int(chr(c) in string.ascii_lowercase)) for c, i in zip([ord(a) for a in password],
range(0xffff)))]])
            with open("passwords/" + str(pw_idx) + "_" + masked_file_name + ".enc",
"wb") as f:
                f.write(rc4(password, generate_key()))
        print("Import complete! Passwords securely stored on disk with your private
key in flag.txt! You may now safely delete flag.txt.")
```

```
else:
    print("This feature is not implemented. Check back in a later update.")
```

Only import functionality is implemented, which is broken in the sense that the encrypted password is stored in the file whose name is just a cipher of the password XD

The complicating looking `masked_file_name` is nothing just a shift cipher by shift `i` at `ith` index.

if the character at index `i` is a digit, `i` is added to it modulo 10

else if the character is a alphabet, it is shifted by `i` modulo 26

```
def decode_filename(file_name):
    num, mask = file_name.split('_')
    file_mask = mask.split('.enc')[0]
    password = ""
    for i in range(len(file_mask)):
        char = file_mask[i]
        if char.isdigit():
            password+=chr( (ord(char)-ord("0")-i+10)%10 + ord('0') )
        else:
            password+=chr( (ord(char)-ord("a")-i+26)%26 + ord('a') )
    return password,int(num)
```

Lets just get all the password, and encrypted password pairs

```
def get_passwords():
    p_ct_list = []
    for file_name in os.listdir('worst-pw-manager/passwords'):
        password, num = decode_filename(file_name)
        with open('worst-pw-manager/passwords/'+file_name,'rb') as enc_file:
            encrypted = enc_file.read()
        p_ct_list.append([num,password,encrypted])
    return p_ct_list
```

Now the real challenge is to recover the key from plaintext/ciphertext pairs.

The key is `cycle(flag_characters)` which means 8 characters of the flag are taken at a time, looping again to start once the flag ends.

This got me into rabbit holes searching for known plaintext key recovery attacks on rc4. After googling a bit, I tried bruteforce on the key since the key is only 8 bytes and the key used to encrypt first block would contain the prefix `flag{`, which makes it only 3 bytes to bruteforce. One could extend this strategy to get key bytes over and over once since somewhere the next 5 characters would be prefix to some 8 byte key block.

Later did I notice that the key generation was buggy

```
def generate_key():
    key = [KeyByteHolder(0)] * 8 # TODO: increase key length for more security?
    for i, c in enumerate(take(flag, 8)): # use top secret master password to encrypt
        all passwords
```

```

    key[i].num = c
return key

```

[KeyByteHolder(0)] * just simply creates an instance KeyByteHolder(0) and generates 8 references to it. The correct way would have been [KeyByteHolder(0) for _ in range(8)]. Given this fact, all the key bytes are actually the last byte repeated 8 times (because of the last assignment)

```

def bruteforce_keylast(pt,ct):
    for i in range(256):
        key = bytearray([i for _ in range(8)])
        if rc4(pt,key) == ct:
            return i

data = get_passwords()
data = sorted(data, key = lambda x:x[0])
eighth_chars = [(ind,bruteforce_keylast(pt,ct)) for ind,pt,ct in data ]

```

And we get a list somewhat

```

['y', 's', 'n', 'n', 'l', 't', 'u', '-', 'i', 'd', 'r', '-', 'a', 'o', 'u', 'g', '-',
'i', 'y', '-', 'f', 'p', 't', 'd', '-', 'i', 'c', 's', '-', 'h', 't', 'a', 'o', 'p',
'i', 'd', 'r', '-', 'a', 'o', 'u', 'g', '-', 'i', 'y', '-', 'f', 'p', 't', 'd', '-',
'i', 'c', 's', '-', 'h', 't', 'a', 'o', 'p', 'p', 's', '}', 'y', 's', 'n', 'n', 'p',
'{', 'i', 'd', 't', 's', 'l', 't', 'u', '-', 'i', 'd', 'r', '-', 'a', 'o', 'u', 'g',
 '-', 'i', 'y', '-', 'f', 'p', 't', 'd', '-', 'i', 'c', 's', '-', 'h', 't', 'a', 'o',
 'p', 'p', 's', '}', 'y', 's', 'n', 'n', 'p', '{', 'i', 'd', 't', 's', 'l', 't', 'u',
 '-', 'i', 'd', 'r', '-', 'a', 'o', 'u', 'g', '-', 'i', 'y', '-', 'f', 'p', 't', 'd',
 '-', 'i', 'c', 's', '-', 'h', 't', 'a', 'o', 'p', 'p', 's', '}', 'y', 's', 'n', 'n',
 'p', '{', 'i', 'd', 't', 's', 'l', 't', 'u', '-', 'i', 'd', 'r', '-', 'a', 'o', 'u',
 'g', '-', 'i', 'y', '-', 'f', 'p', 't', 'd', '-', 'i', 'c', 's', '-', 'h', 't', 'a',
 'o', 'p', 'p', 's', '}', 'y', 's', 'n', 'n', 'p', '{', 'i', 'd', 't', 's', 'l', 't',
 'u', '-', 'i', 'd', 'r', '-', 'a', 'o', 'u', 'g', '-', 'i', 'y', '-', 'f', 'p', 't',
 'd', '-', 'i', 'c', 's', '-', 'h', 't', 'a', 'o', 'p', 'p', 's', '}', 'y', 's', 'n',
 'n', 'p', '{', 'i', 'd', 't', 's', 'l', 't', 'u', '-', 'i', 'd', 'r', '-', 'a', 'o',
 'u', 'g', '-', 'i', 'y', '-', 'f', 'p', 't', 'd', '-', 'i', 'c', 's', '-', 'h', 't',
 'a', 'o', 'p', 'p', 's', '}', 'y', 's', 'n', 'n', 'p', '{', 'i', 'd', 't', 's', 'l',
 't', 'u', '-', 'i', 'd', 'r', '-', 'a', 'o', 'u', 'g', '-', 'i', 'y', '-', 'f', 'p',
 't', 'd', '-', 'i', 'c', 's', '-', 'h', 't', 'a', 'o', 'p', 'p', 's', '}', 'y', 's',
 'n', 'n', 'p', '{', 'i', 'd', 't', 's', 'l', 't', 'u', '-', 'i', 'd', 'r', '-', 'a',
 'o', 'u', 'g']

```

Which is the list over eighth characters modulo the flag length, which is yet unknown. All we need to do is loop over the possible flag lengths and check if it contains flag

```

for flag_len in range(9,50):
    flag = bytearray(flag_len)
    for ind,key_char in eighth_chars: #index number specified by the index of password
file
        starting_ind = (ind*8)%flag_len #starting index of password string
        flag[(starting_ind+8)%flag_len] = key_char #found password
    if b'flag' in flag:
        print(flag.decode())
        break

```

flag-crypto'is'stupid'and'python'is'stupid"

Yet another fun challenge! Good job redpwn guys!

| Interactive controls are omitted from this PDF. See the canonical HTML version.